Lab 6: The Unix Privilege Model, Setuid and Sudo

Computing Systems Security, 2026-27 MCS and MEI, University of Minho

Vítor Francisco Fonte vff@di.uminho.pt

Block 2 asked who a process is and how it proves it. Block 3 asks the next question: once identity is established, **what can a process actually do**, and how does a process legitimately come to hold more privilege than the user who started it? This lab is about the two everyday mechanisms that grant exactly that, setuid binaries and `sudo`, and about how both become privilege-escalation paths when they are configured carelessly. You already met both in earlier weeks, the undue setuid binary in Lab 1's inventory and the permissive `sudo` rule in your Lab 2 model, where you recorded it with `sudo -l -U svc-report` as a permitted command whose reach you had not yet measured. Here you measure what each actually reaches: escalate through them **as an unprivileged non-owner would**, then fix both and show the escalation is closed.

This is the first lab of the attacker-facing half of the course. From here on, you do not merely describe a weakness, you exploit it and then close it.

Safety. You have full `sudo` on this VM by design, so the exercises are careful to make you escalate **without** using your own privilege: you will act as the unprivileged service account `svc-report` from Lab 1, and prove each escalation from *there*. Snapshot `lab6-clean` before you start. Nothing here can lock you out, but the sudo edits in Part D should be reverted from your own shell or the snapshot.

Contents

- Learning Objectives
- Before the lab (at home, ~30 min)
- Part A: The privilege model in motion
- Part B: Enumerate the setuid surface
- Part C: Exploit the setuid binary
- Part D: Read and exploit the sudo policy
- Part E: Close both escalations
- If it does not work
- Reflection
- References
- Deliverables

Learning Objectives

By completing this lab, students will be able to:

1. Explain the real, effective and saved user IDs and the transitions a setuid binary causes (the definitions came in Lab 3; here you see the transitions happen).
2. Find setuid and setgid binaries on a system and reason about which are legitimate and which are a design smell.
3. Exploit an unsafe setuid binary to gain privilege from an unprivileged account, and explain precisely why it works.
4. Read a `sudo` policy as an attack surface and exploit a permissive rule.
5. Close both escalations and verify they are closed, and state the principle each fix applies.

The setuid and sudo findings are **capstone link 2 (host escalation)**.

What from this lab is assessed. Two things, both handed in with **Lab report 3** at the end of the block (Lab 8): your answers to the preparation questions **P1 and P2**, and your **Reflection**. The exercises themselves

are not handed in. Keep your notes and your two escalation transcripts: Lab 8 and the capstone's host-escalation link build on them.

Before the lab (at home, ~30 min)

Boot the VM and snapshot `lab6-clean`. Re-read Lecture 6 (the privilege model) if you have already had it, and your Lab 1 notes on the seeded surface, in particular the setuid `backup-tool` and the `svc-report` account. Re-open the **Lab 2 model** as well: the DFD, the STRIDE table and the attack tree you built there named `svc-report`'s one narrow `sudo` grant as *a permitted command whose reach you had not yet measured*. Part D of this lab is where you measure it, so bring that open question with you rather than starting the account from scratch.

Open a shell **as the unprivileged attacker**, not as yourself. From your `student` shell:

```
sudo -u svc-report -s      # drop to the service account; this is your attacker
 ↪  foothold
id                         # confirm you are svc-report, uid != 0 (svc-report has
 ↪  ONE narrow sudo grant, which Part D is about; it is not a general sudoer)
```

Everything in Parts C and D is done from this `svc-report` shell. When a step says "as yourself", it means your ordinary `student` shell in another terminal.

Preparation questions (submit with Lab report 3)

- **P1.** In one sentence each, define the real UID and the effective UID of a process, and say which one the kernel checks for a file-permission decision.
- **P2.** What does the setuid bit on an executable do to a process's effective UID when it runs? Whose privilege does the process then carry?

Part A: The privilege model in motion

Exercise 1. Watch a setuid transition happen with a program you trust: `passwd`, which is setuid root so it can write `/etc/shadow`.

```
ls -l /usr/bin/passwd                  # note the 's' in the permission bits
# in one shell, start passwd and leave it at the prompt; in another:
ps -o pid,ruser,euser,cmd -C passwd    # real user vs effective user
```

1a. `passwd` was started by an unprivileged user but runs with effective UID 0. Name the mechanism, and explain why `passwd` *needs* this while an ordinary editor does not. What would go wrong if a setuid program like `passwd` kept effective UID 0 for its whole run, instead of dropping it the moment its privileged work was done?

Part B: Enumerate the setuid surface

Exercise 2. Find every setuid and setgid binary on the system, as an attacker profiling the box would:

```
find / -perm -4000 -type f 2>/dev/null     # setuid
find / -perm -2000 -type f 2>/dev/null     # setgid
```

2a. Record the list. Most entries are legitimate system tools (`passwd`, `sudo`, `mount`, `su`). One is not: the `backup-tool` from Lab 1, installed setuid root. State, in general terms:

- why a setuid binary is a **design smell**: an amount of privilege that must be justified rather than assumed;
- why a *custom* setuid binary deserves far more suspicion than one the distribution ships.

Part C: Exploit the setuid binary

The seeded `backup-tool` is setuid root and is, in effect, a thin wrapper that copies a file you name to a backup location, running as root. It does not drop privilege and it does not constrain the paths it touches. That combination is the whole vulnerability.

Exercise 3. As `svc-report`, inspect and then abuse it.

```
ls -l /usr/local/bin/backup-tool              # setuid-root (-rwsr-xr-x root root)
# it is a setuid copy of cp, so it takes a SOURCE and a DESTINATION:
/usr/local/bin/backup-tool /etc/hostname /tmp/hn && cat /tmp/hn   # a benign call
```

3a. The tool runs as root and reads or writes paths you choose, with no restriction. Use it to obtain something `svc-report` must not have. Two paths are open, and you should demonstrate **one** and describe the other:

- **read** a root-only file by having the tool copy it somewhere you can read (for example `backup-tool /etc/shadow /tmp/leak`, then `cat /tmp/leak`; `svc-report` cannot read `/etc/shadow` directly);
- **write** to a root-owned location to plant or alter a file (for example dropping a key into a root-writable path).

`backup-tool` takes two arguments. It is a setuid-root copy of `cp`, so the call is **source dest**; give it one argument and you get cp's `missing destination file operand after '/etc/shadow'`, not a usage banner. (With *no* argument at all it says `missing file operand`; the two messages are different, and it is the one-argument form you will hit.) That is the whole escalation: `svc-report` cannot read `/etc/shadow` directly and gets Permission denied, but `backup-tool /etc/shadow /tmp/leak` copies it as root, and the hashes are readable in the copy.

3b. Explain the escalation precisely, in terms of Part A's model: whose effective UID did the copy run under, which check did the kernel make and against which identity, and why the tool's failure to drop privilege or validate paths is what made you root-equivalent. This is the sentence that shows you understand setuid, not just that you ran a command.

Part D: Read and exploit the sudo policy

`sudo` grants privilege by policy rather than by a file bit. The policy in `/etc/sudoers` (and `/etc/sudoers.d/`) is only as safe as its narrowest rule.

Exercise 4. The machine ships a `sudoers.d` drop-in that lets `svc-report` run `find` without a password, meant so it can search its own logs. As yourself, read the policy; then, as `svc-report`, exploit it.

```
# as student (your own shell):
sudo cat /etc/sudoers.d/svc-report
# as svc-report:
sudo -ln                                      # what may I run without a password?
# find has -exec, so a permitted find is a root shell:
sudo find /etc/hostname -exec id \;           # runs id as root: uid=0
```

The rule this exercise turns on. The seeded sudoers entry is `svc-report ALL=(ALL) NOPASSWD: /usr/bin/find`, the kind of "let the service search its own logs" grant that looks harmless on review. It is not. `find` has an `-exec` action that runs a command for each file it matches. Under this rule that command runs as root: `sudo find <anypath> -exec id \;` prints `uid=0(root)`, `-exec /bin/sh \;` opens an interactive root shell, and `-exec <cmd> \;` runs any command at all as root.

4a. The rule looks narrow but is not. Using only what `sudo -ln` says you may run, obtain root-level access (here, `find`'s `-exec`). Explain the technique and name the general class of the mistake (a permitted command that can execute other programs).

4b. The rule named a single command, yet it gave you a root shell. Explain why permission to run ”one command” is so often not a single, bounded power: what must be true of a command for the right to run it to hand you everything that command can reach?

Part E: Close both escalations

Exercise 5. Fix the setuid binary. There is more than one candidate answer, and **one of them does not work**; finding out which, and why, is the exercise.

- remove the setuid bit entirely if the tool does not need root;
- replace ambient root with the **specific capability** it needs (a forward pointer to Lab 7) rather than full setuid root;
- or constrain what it may touch.

5a. Apply the fix you judge correct, then **prove** the escalation from Part C no longer works as `svc-report`. State which principle your fix applies and what you gave up to close the hole.

⚠ **If you took the capability route, measure it before you believe it.** ”Grant one capability instead of all of root” is the right instinct and it is what Lab 7 is about, but ask which capability this tool would need. `backup-tool` is a copy of `cp` whose job is to read any file and write it somewhere, so the DAC checks it must bypass are the same ones the attack needs. Measured on this image, with the setuid bit **removed**:

grant on `backup-tool`	`backup-tool /etc/shadow /tmp/leak`
`cap_dac_read_search+ep`	**succeeds**: the file is copied and you can read the copy
`cap_dac_override+ep`	**succeeds**: the same
nothing at all	fails, `Permission denied`

⚠ **And read the two capabilities as different, because they are.** `CAP_DAC_READ_SEARCH` bypasses read and directory-search checks; `CAP_DAC_OVERRIDE` bypasses read, write **and** execute (the last only where an execute bit is already set). Both leave the *read* escalation of 3a wide open. Only `CAP_DAC_OVERRIDE` also leaves the *write* route open. So the narrower grant did shrink the blast radius, and did **not** close the hole 5a asked you to close: those are two different things and the exercise turns on the difference.

5b. Which capability did you attach, and what exactly did it still allow? If you did not take the capability route, run it now anyway and record the result: an option that looks correct and is not is worth more to you than the one that works.

5c. **The real question of Part E, so answer it:** for ”give it one capability instead of root” to *close* a hole rather than merely narrow it, what must be true of the tool? (That is: what does `backup-tool` legitimately need to do, and does the attack need the same power?) Then say which of the three routes above can actually close this hole, and what it costs.

Exercise 6. Fix the sudo policy. Rewrite the drop-in so the account can still do its legitimate job (if any) but can no longer escalate. Then prove Part D's technique fails.

6a. Show the escalation is closed. Explain why your rewrite removes the capability without removing the intended function, or argue that the intended function cannot be granted safely and should be removed.

Restore `lab6-clean` when finished, or confirm both fixes are in place if you are keeping the machine.

If it does not work

One thing catches almost everyone, and it is not a permissions problem.

What you see	What to do
`backup-tool /etc/shadow` answers `missing destination file operand after '/etc/shadow'`	`backup-tool` is a setuid copy of `cp`, so it needs a **source and a destination**: `backup-tool /etc/shadow /tmp/leak`, then read `/tmp/leak`.

If none of these matches, read the error message in full first: it usually names the layer that refused, and that is most of the answer. Then ask your lab teacher, in the session or by email.

Reflection

Your reflection goes into Lab report 3, as an appendix, beside the reflections from the other labs in this block. They are assessed with the report: every one has to be there, each answering its questions with a position you argue from what you did in the lab.

In a short piece (200-300 words):

1. Setuid and sudo solve the same problem, letting an unprivileged user perform a privileged action, in two different ways: one binds privilege to a *binary*, the other to a *policy*. Compare their failure modes. Which is easier to reason about, and which did you find easier to exploit?
2. Both weaknesses you exploited were legitimate mechanisms used carelessly, not bugs in the kernel. This is the recurring shape of the course (the world-writable script run by root's cron job in Lab 1, the agent-forwarding default in Lab 5). What does that suggest about where an operator's attention should go?
3. Capstone link. The target you will attack in the capstone ships one of these two weaknesses as its host-escalation step. Which of your two fixes would you apply first there, and why?

References

- `credentials(7)` and `execve(2)` (setuid transitions); `setuid(2)`, `setresuid(2)`.
- `sudoers(5)` and `sudo(8)`; the `sudo -l` policy listing.
- GTFOBins, https://gtfobins.github.io/ (shell escapes from permitted binaries), for the sudo exploitation technique.
- W. Du, *Computer Security: A Hands-on Approach*, 3rd ed., 2022, the Set-UID chapter.

Deliverables

There is **no separate report for this lab**; keep your notes and your two escalation transcripts. Block 3's assessed deliverable is **Lab report 3**, produced in Lab 8, and it collects this lab's preparation questions P1-P2 (required). Bring your setuid and sudo escalation transcripts to Lab 8: the capstone's host-escalation link builds directly on them.